Streszczenie

TODO

Abstract

TODO

Spis treści

1 Wstęp **3**

2 Notacja Ficha i jej interpretacja **4**

3 Ustawienie modelu **7**

4 Parsowanie dowodów w notacji Ficha **8**

5 Generowanie korpusu **8**

6 Trenowanie i kożystanie z modelu (NanoGPTBundle i te wszystkie funkcje z nim związane !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!, podkreślam WSZYSTKIE(liczenie prawdopodobieństwa, dodawanie tokenu, linii, trenowanie, dotrenowanie, ładowanie)!!!!) **8**

7 3 strategie **8**

1 Wstęp

Dedukcja naturalna jest metodą dowodzenia twierdzń logicznych przez wychodzenie z pewnych założeń i modyfikowanie ich za pomocą ściśle zdefiniowanych reguł wnioskowania.

Definicja (sekwent)

Sekwentem nazywamy wyrażenie postaci

$$\Gamma \vdash \varphi$$

gdzie Γ jest skończonym zbiorem formuł (zapisywanych po przecinku i bez nawiasów klamrowych) zwanym *kontekstem*, a φ jest formułą zwaną *tezą sekwentu*. Sekwent $\Gamma \vdash \varphi$ interpretujemy jako stwierdzenie, że jeśli założymy, że wszystkie formuły z Γ są prawdziwe, da się dowieść φ. W rachunku zdań w logice klasycznej, który będzie jedyną logiką w której się będziemy poruszać, zdanie prawdziwe we wszystkich wartościowaniach jest dowodliwe, tak więc $\Gamma \vdash \phi$ jest równoważne stwierdzeniu jeżeli wszystkie formuły kontekstu Γ są prawdziwe to φ jest prawdziwe.

Definicja (reguła wnioskowania)

Regułą wnioskowania nazywamy schemat postaci

$$\frac{S_1 \qquad S_2 \qquad \dots \qquad S_n}{S}(R)$$

gdzie $S_1, \dots, S_n, S$ są sekwentami. Sekwenty $S_1, \dots, S_n$ nazywamy *przesłankami reguły*, natomiast sekwent S nazywamy *konkluzją reguły*. R to z kolei *indeks reguły*, który służy do jej identyfikacji.

Dowody w dedukcji naturalnej można utożsamiać z drzewami, których węzłami są sekwenty, a krawędzie są indeksowane indeksami reguł wnioskowania.

W nieniejszej pracy skupimy się na dowodzeniu twierdzeń w logice klasycznej z następującymi spójnikiami logicznymi: koniunkcją ($\wedge$), alternatywą ($\vee$), negacją ($\neg$) oraz implikacją ($\rightarrow$). regułami wnioskowania:

$$\frac{}{\varphi \vdash \varphi}\ (\text{Ass})$$

$$\frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \rightarrow \psi}\ (\rightarrow I)$$

$$\frac{\Gamma, \varphi \vdash \bot}{\Gamma \vdash \neg\varphi}\ (\neg I)$$

$$\frac{\Gamma_1 \vdash \varphi \qquad \Gamma_2 \vdash \psi}{\Gamma_1, \Gamma_2 \vdash \varphi \wedge \psi}\ (\wedge I)$$

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi}\ (\vee I_1)$$

$$\frac{\Gamma \vdash \psi}{\Gamma \vdash \varphi \vee \psi}\ (\vee I_2)$$

$$\frac{}{\vdash \top}\ (\top I)$$

$$\frac{\Gamma_1, \varphi \vdash \psi \qquad \Gamma_2, \psi \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \varphi \leftrightarrow \psi}\ (\leftrightarrow I)$$

$$\frac{\Gamma_1 \vdash \varphi \leftrightarrow \psi \qquad \Gamma_2 \vdash \psi}{\Gamma_1, \Gamma_2 \vdash \varphi}\ (\leftrightarrow E_2)$$

$$\frac{\Gamma \vdash \neg\neg\varphi}{\Gamma \vdash \varphi}\ (\neg\neg)$$

$$\frac{\Gamma \vdash \varphi}{\Gamma, \psi \vdash \varphi}\ (\text{W})$$

$$\frac{\Gamma_1 \vdash \varphi \rightarrow \psi \qquad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \psi}\ (\rightarrow E)$$

$$\frac{\Gamma_1 \vdash \neg\varphi \qquad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \bot}\ (\neg E)$$

$$\frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \varphi}\ (\wedge E_1)$$

$$\frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \psi}\ (\wedge E_2)$$

$$\frac{\Gamma_1 \vdash \varphi \vee \psi \qquad \Gamma_2, \varphi \vdash \rho \qquad \Gamma_3, \psi \vdash \rho}{\Gamma_1, \Gamma_2, \Gamma_3 \vdash \rho}\ (\vee E)$$

$$\frac{\Gamma \vdash \bot}{\Gamma \vdash \varphi}\ (\bot E)$$

$$\frac{\Gamma_1 \vdash \varphi \leftrightarrow \psi \qquad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \psi}\ (\leftrightarrow E_1)$$

$$\frac{\Gamma, \neg\varphi \vdash \bot}{\Gamma \vdash \varphi}\ (\text{RAA})$$

$$\frac{}{\vdash \varphi \vee \neg\varphi}\ (\text{TND})$$

Tabela 1: Reguły wnioskowania rozważanego systemu

Oto przykładowy dowód w dedukcji naturalnej w notacji drzewiastej:

$$\dfrac{\dfrac{\dfrac{}{a \vee b \vdash a \vee b}\,(\text{Ass}) \qquad \dfrac{\dfrac{}{a \vdash a}\,(\text{Ass})}{a \vdash b \vee a}\,(\vee I_2) \qquad \dfrac{\dfrac{}{b \vdash b}\,(\text{Ass})}{b \vdash b \vee a}\,(\vee I_1)}{a \vee b \vdash b \vee a}\,(\vee E)}{\vdash (a \vee b) \to (b \vee a)}\,(\to I)$$

Dużą zaletą dedukcji naturalnej jest to, że niejako symuluje ona sposób, w jaki ludzie przeprowadzają wnioskowanie, zarówno w matematyce, jak i w codziennym życiu. Dowody mają swoją drzweiastą strukturę, którą można łatwo analizować. Inną zaletą tej metody jest jej uniwersalność. Dodając odpowiednie reguły wnioskowania możemy otrzymać dedukcję naturalną dla różnych systemów logicznych np logiki pierwszego i drugiego rzędu, czy logiki modalnej. Jeszcze jedną zaletą jest to, że w przypadku większości użytecznych dowodów jesteśmy w stanie uniknąć wykładniczo dużej złożoności wielu innych systemów dowodzenia sprowadzających się do rozpatrzenia wszystkich możliwych przypadków. Z drugiej strony dedukcja naturalna jest trudna w zaitomatyzowaniu. Zwykle dowód przeprowadzamy z góry na dół, zaczynając od założeń i kończąc na tezie. Jednak nie zawsze jasne jest, którą regułę wnioskowania zastosować i co gorsza wyniki zastosowania niektórych reguł mogą zawierać podformuły których nie ma w przesłankach, lub jej wybór nie jest jednoznaczny. Np używając reguły

$$\dfrac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi}\,(\vee I_1)$$

w przesłance nie mamy informacji o formule ψ, która pojawi się w konkluzji. Jednak podobieństwo dedukcji naturalnej do ludzkiego sposobu wnioskowania sprawia, oraz pewien dający się zauważyć porządek w zapisie dowodów sprawia, że gdzie klasyczna implementacja dedukcji naturalnej jest trudna, wydaje się ona dobrze nadawać do implementacji z użyciem metod sztucznej inteligencji, w tym modeli językowych o arhirtekturze transformerów.

W niniejszej pracy przedstawimy implementację automatycznego systemu dowodzenia twierdzeń w logice klasycznej przy użyciu odpowiednio wytrenowanego modelu nano-GPT oraz przedstawimy wyniki jego działania i jak dobrze radzi sobie z przeprowadzaniem dowodów. Konstrukcja systemu będzie opisana krok po kroku, zaczynając od zaprogrogramowania logicznej struktury systemu, tłumaczenia zapisu dowodu w formie tekstowej na strukturę logiczną i sprawdzanie poprawności dowodu, generacja korpusu treningowego, trenowanie modelu oraz kolejne etapy konstrukcj samego systemu dowodzenia(wybór możliwych linii, równoległość prowadzenia kilku ścieżek rozumowania, itp).

2 Notacja Ficha i jej interpretacja

Notacja Ficha jest sposobem zapisu z dowodów dedukcji naturalnej w formie tekstowej. Jest on wygodny, ponieważ klasyczna metoda zapisywania drzew dowodu potrafi zajmować bardzo dużo miejsca i trudno użyć do niej modeli językowych. Notacja Ficha jest z kolei metodą zapisu dowodu w formie tekstu, gdzie każda linia w specyficzny sposób koduje sekwent uzyskany przez używanie reguł do sekwentów kodowanych w poprzednich liniach.

Przyjmijmy, następującą funkcję:

`Fitch(i)` := sekwent kodowany przez linię o indeksie `i`

oraz oznaczenie

`s.conclusion` := teza sekwentu `s`

`s.assumptions` := kontekst sekwentu `s`

Linia dowodu w notacji Ficha można podzielić na 4 części: indeks linii, konkluzja sekwentu kodowanego przez tą linię, nazwa reguły wnioskowania i indeksy linii które używamy przy użyciu tej reguły.

Poniżej przedstawimy interpretację poszczególnych reguł wnioskowania w notacji Ficha, a także ich odpowiedniki w notacji drzewiastej.

Notacja Fitcha	**Notacja drzewiasta**
i. ϕ assumption	$\dfrac{}{\phi \vdash \phi}\ (\mathrm{Ass})$
i. ϕ weakening, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (W)$
j. ϕ ... ... k. ψ ... ... i. $\phi \to \psi$ →-introduction, j-k	$\dfrac{\texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\to I)$
j. ϕ ... ... k. $\bot$ ... ... i. $\neg\phi$ ¬-introduction, j-k	$\dfrac{\texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\neg I)$
j. ϕ ... ... k. $\phi \to \psi$ ... ... i. ψ →-elimination k, j	$\dfrac{\texttt{Fitch(k)} \quad \texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\to E)$
j. $\neg\phi$ ... ... k. ϕ ... ... i. $\bot$ ¬-elimination, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\neg E)$
j. ϕ ... ... k. ψ ... ... i. $\phi \wedge \psi$ ∧-introduction, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\wedge I)$
j. ϕ ... ... i. $\phi \vee \psi$ ∨-introduction1, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\vee I_1)$
j. ψ ... ... i. $\phi \vee \psi$ ∨-introduction2, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\vee I_2)$
i. $\top$ ⊤-introduction	$\dfrac{}{\vdash \top}\ (\top I)$
j. $\phi \wedge \psi$ ... ... i. ϕ ∧-elimination1, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\wedge E_1)$
j. $\phi \wedge \psi$ ... ... i. ψ ∧-elimination2, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\wedge E_2)$

j. $\phi \vee \psi$ ... ... k. ρ ... ... l. ρ ... ... i. ρ $\vee$-elimination, j, k, l	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)} \quad \texttt{Fitch(l)}}{\texttt{Fitch(i)}}\ (\vee E)$
j. $\perp$ ... ... i. ϕ $\perp$-elimination, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\perp E)$
j. ψ ... ... k. ϕ ... ... i. $\phi \leftrightarrow \psi$ $\leftrightarrow$-introduction, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\leftrightarrow I)$
j. $\phi \leftrightarrow \psi$ ... ... k. ϕ ... ... i. ψ $\leftrightarrow$-elimination1, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\leftrightarrow E_1)$
j. $\phi \leftrightarrow \psi$ ... ... k. ψ ... ... i. ϕ $\leftrightarrow$-elimination2, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\leftrightarrow E_2)$
j. $\neg\phi$ ... ... k. $\perp$ ... ... i. ϕ RAA, j-k	$\dfrac{\texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\text{RAA})$
j. $\neg\neg\phi$ ... ... i. ϕ $\neg\neg$-elimination, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\neg\neg)$
i. $\phi \vee \neg\phi$ TND	$\dfrac{}{\vdash \phi \vee \neg\phi}\ (\text{TND})$

Warto zwuważyć, że aby dowody były poprawne muszą być spełnione odpowiednie warunki dotyczące sekwentów do których się odwołujemy, wymuszone przez reguły wnioskowania. Np dla napisu:

j. $\phi \vee \psi$ ...
...
k. ρ ...
...
l. ρ ...
...
i. ρ $\vee$-elimination, j, k, l

musi być spełnione
$\phi \in$ `Fich(k).assumptions` oraz $\psi \in$ `Fich(l).assumptions`.

3 Ustawienie modelu

Bazą naszego systemu są klasy `Relation` o `Variable`. `Relation` reprezentuje relację, zawiera akceptowalne arności, nazwę relacji oraz metodę `toString`, która zwraca jej reprezentację tekstową. W naszym modelu używamy jedynie zmiennych zdaniowych, więc wystarczyłoby jedynie użycie klasy reprezentacej zmienną jednak użycie klasy `Relation` pozwala na łatwe rozszerzenie modelu w przyszłości o predykaty wieloargumentowe. Dla uproszczenia zapisu dodajemy jednak dodajemy fukcję która wczytuje/wypisuje jedynie nazwy zmiennych i zmienną `x` interpretuje jako `'(x)`.

Klasa `Variable` reprezentuje zmienną, którą dajemy jako argument do relacji. Zawiera ona nazwę zmiennej oraz metodę zwracającą jej reprezentację tekstową.

Kolejną klasą jaką definiujemy jest klasa `Formula`, która reprezentuje formułę logiczną. Podstawową i najprostszą formułą jest formuła `Atom`, której implementacja zawiera nazwę relacji którą reprezentuje oraz listę argumentów na których ona zachodzi. Poza formułą atomową mamy również klasy reprezentujące negację, koniunkcję, alternatywę, implikację oraz równoważność. Klasy te zawiera swój typ oraz dwu lub jedno elementową listę podformuł w polu `Interior`.

Kolejną zaimplementowaną klasą jest klasa `Context`, która reprezentuje kontekst dowodzenia, czyli zbiór aktualnie przyjętych formuł. Z powodów które zostaną wyjaśnione w dalszej części pracy `Context` podzieliliśmy, na dwie listy: `base_context` czyli formuły które przyjmujemy za prawdziwe w kontekście całego segmentu dowodu, oraz `additional_context`, czyli formuły, które mogą zmieniać się w każdej linii dowodu. Obiekty klasy `Context` możemy sumować ze sobą jak zbiory metodą `union`, a także odejmować element, lub inny kontekst metodą `remove`. Operacje te wpływają jednak jedynie na `additional_context`, zaś `base_context` pozostaje bez zmian. Aby dwa konteksty można było od siebie odjąć lub dodać, muszą mieć takie samo `base_context`, w przeciwnym wypadku zostaje zwrócony `TypeError("Bad arguments")`.

Sekwent reprezentuje klasa `LittleProblem`, która składa się z kontekstu w polu `assumptions` oraz tezy w polu `conclusion`.

Kolejną klasą jest klasa `Rule`, która reprezentuje regułę wnioskowania. Dziedziczą po niej klasy

- `Assumption`
- `Weakening`
- `ImplicationIntroduction`
- `NegationIntroduction`
- `ImplicationElimination`
- `NegationElimination`
- `ConjunctionIntroduction`
- `DisjunctionIntroduction1`
- `DisjunctionIntroduction2`
- `TruthIntroduction`
- `ConjunctionElimination1`
- `ConjunctionElimination2`
- `DisjunctionElimination`
- `LieElimination`
- `IffIntroduction`
- `IffElimination1`
- `IffElimination2`
- `RAA`

- `NegationOfNegation`
- `TND`

, które reprezentują odpowiednie reguły wnioskowania. Każda z tych klas zawiera metodę `top_down`, która bierze listę sekwentów będącymi przesłankami reguły. Niektóre reguły wymagają doatkowych niejednoznacznych argumentów. Np zgodnie z notacją w Tabeli 1 reguła (Ass) wymaga znajomości formuły ϕ która może być dowolna, z kolei zastosowanie reguły (RAA), wymaga znajomości ϕ, która jest dowolną formułą, której zanegowanie występuje w kontekście sekwentu podanego jako przesłanka. W takim wypadku podajemy te niejednoznaczne formuły, jako dodatkowy argument oprócz przesłanek. Również `base_context` który zakładamy jako prawdziwy dla całego dowodu nie koniecznie da się wydedukować z listy przesłanek (chodzi o sytuacje dla reguł (Ass), (TND) i $(\top I)$, gdzie lista przesłanek jest pusta. Wtedy konieczne jest podanie również bazowego kontekstu w formie listy formuł. Oprócz generacji konkluzji reguły `top_down` sprawdza również, czy wszystkie podane argumenty są poprawne. Po pierwsze chodzi o to, czy `base_context` kontekstów przesłanek są takie same, czy listy przesłanek mają odpowiednie długości i czy ich elementy są klasy `LittleProblem`. Kolejną kwestią jest to, czy argumenty, można faktycznie podstawić do danej reguły. Np czy dla reguły $\neg E$ teza pierwszej przesłanki jest negacją tezy drugiej przesłanki, lub czy dla reguły RAA $\neg\phi$ jest w kontekście przesłanki, przy czym ϕ jest przechowywane w argumencie `phi` funkcji `top_down` klasy `RAA`. Jeżeli argumenty nie spełniają odpowiednich warunków, zostaje zwrócony błąd `TypeError("Bad arguments")`.

4 Parsowanie dowodów w notacji Ficha

W celu kontroli poprawności dowodu w notacji Fitcha, jest on w programie parsowany do reprezentującej go struktury logicznej. Dowody reprezentujemy jako słownik, którego kluczami są indeksy linii, a wartościami obiekty klasy `structure_of_line_top_down`. Klasa ta zawiera pola `Key`, które reprezentuje indeks linii, `args_keys`, które reprezentują indeksy linii, które są używane jako przesłanki, `rule`, które reprezentuje regułę wnioskowania, której używamy oraz, `LP`, które reprezentuje sekwent który koduje dana linia. Wartość `LP` jest generowana przez regule `top_down` zastosowaną dla przesłanek reprezentowanych przez `args_keys` i pozostałych argumentów Funkcja `__init__` tej klasy bierze za argument `Key`, `arg_keys`, `rule`, a także `Proof_so_far`, które zawiera sparsowany do formy słownika dotychczasowy dowód, który jest konieczny gdy chcemy odwoływać się do przesłanek, oraz `formula_text`, która jest wyciągniętą z linii formułą w notacji infiksowej, z której będzie niekiedy konieczne dedukować argumenty nie będące przesłankami w regule `top_down`. Ostatnim argumentem dla funkcji `__init__` jest `BaseCntext`, który jest przyjmowanym w danym dowodzie kontekstem bazowym.

Funkcja `parseProof` tworzy słownik, w którym każdemu indeksowii linii jest przypisany odpowiedni obiekt typu `structure_of_line_top_down`. Funkcja ta jako argumenty przyjmuje dowód w notacji Fitcha w formie tekstowej, oraz używany w dowodzie kontekst bazowy. Funkcja ta wczytuje dowód linia po linii i aktualizuje słownik. Sprawdza również, czy formuła będąca konkluzją pola `LP` obiektu klasy `structure_of_line_top_down` odpowiadająca danej linii jest taka sama jak formuła zapisana w zapisie tekstowej tej linii. Jeżeli tak nie jest oznacza to, że dowód jest niepoprawny i zostaje zwrócony błąd `TypeError("Bad arguments")`.

5 Generowanie korpusu

6 Trenowanie i kożystanie z modelu (NanoGPTBundle i te wszystkie funkcje z nim związane !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!, podkreślam WSZYSTKIE(liczenie prawdopodobieństwa, dodawanie tokenu, linii, trenowanie, dotrenowanie, ładowanie)!!!!)

7 3 strategie

Celem tej pracy nie jest jedynie skonstruowanie modelu językowego generującego dowody w dedukcji naturalnej, lecz także zbudowanie systemu, który w pewnym stopniu naśladuje sposób, w jaki człowiek

dochodzi do poprawnego dowodu. Sam model językowy może generować ciągi tekstu przypominające formalny dowód, jednak nie gwarantuje to ich poprawności logicznej. Z tego powodu każdy wygenerowany krok musi być na bieżąco weryfikowany przy użyciu opisanej wcześniej funkcji `parseProof`.

Drugim problemem jest to, że konstruowanie dowodu rzadko ma charakter całkowicie liniowy. W praktyce często wymaga ono licznych prób, cofania się i wybierania innych ścieżek rozumowania, zanim zostanie znaleziony zapis końcowy. Dlatego druga wersja systemu rozszerza prostą generację kolejnych linii o mechanizm przeszukiwania przestrzeni możliwych kroków dowodowych.

Wreszcie, dla trudniejszych twierdzeń samo lokalne przeszukiwanie kolejnych kroków może okazać się niewystarczające. System może trafiać na ślepe uliczki, ponieważ na początkowym etapie dowodu nie zawsze wiadomo, która strategia doprowadzi do celu. Z tego względu trzecia wersja systemu wprowadza możliwość rozbijania problemu na prostsze podproblemy, które następnie mogą zostać połączone odpowiednią regułą wnioskowania w rozwiązanie problemu wyjściowego.

System był rozwijany etapami. Najprostsza wersja zajmuje się wyłącznie generowaniem kolejnych linii dowodu i sprawdzaniem ich poprawności. Druga wersja dodaje mechanizm przeszukiwania metodą prób i błędów. Ostateczna wersja rozszerza ten schemat o dekompozycję celu na podproblemy. Każda kolejna wersja wykorzystuje poprzednią jako komponent składowy. W dalszej części rozdziału omawiam kolejno wszystkie trzy strategie.

Pierwsza wersja systemu zawiera się w funkcji `silliest_generate_proof`. Jej argumenty to `bundle` będący obiektem klasy `NanoGPTBundle` którym będziemy generować tekst dowodu, `prompt` który jest zmienną tekstową, która koduje twierdzenie, które mamy udowodnić, i ewentualnie początek dowodu, zmienna `temperature` typu `float`, która pozwala kontrolować poziom determinizmu generacji oraz zmienna `max_new_lines` typu `int`, która określa ile linii dowodu chcemy wygenerować i zmienna `max_failed_attempts`, która oznacza ile maksymalnie kolejnych nieudanych prób dopisania linii dowodu akceptujemy. Każdy prompt zaczyna się od słów `Prove that:` a następnie podana jest formuła rachunku zdań w formie infiksowej. Jeżeli kontekst bazowy w generowanym dowodzie ma być niepusty to w drugiej linii znajdują się słowa: `assuming that:` po których wypisana jest pierwsza z formuł bazowego kontekstu. Kolejne formuły są wypisywane po przecinku i każda w nowej linii. Następnie w kolejnych liniach jest dopisywany dowód.

Dowód dopisujemy w pętli, która kończy się kiedy ostatnia linia koduje sekwent, który chcemy udowodnić, liczba nowych linii przekracza `max_new_lines` lub ilość kolejnych nieudanych prób dopisania linii dowodu przekracza `max_failed_attempts`. W każdej iteracji pętli próbujemy dodać jedną linię dowodu. Linia jest odrzucana jeżeli występują w niej zmienne inne niż w docelowym sekwencie, linia nie spełnia warunków składniowych lub jeśli parsowanie dowodu z nową linią zwraca błąd. Jeżeli linia nie jest odrzucana jest dopisywana do końca dowodu i zerowany jest licznik odrzucanych linii. Funkcja zwraca `prompt_str` z dopisanymi liniami dowodu (linie te nie muszą tworzyć pełnego dowodu tezy, ale intuicyjnym celem tej funjkcji jest po prostu zrobienie pewnego postępu w dowodzie).

Druga wersja systemu jest zaimplementowana w funkcji `find_proof_little_smarter`. Idea tej funkcji sprowadza się do tego, że zamiast tworzyć jeden dowód, tworzymy listę potencjalnych prób dowodu (na początku jest w niej jedynie prompt z zakodowanym problemem do udowodnienia) i w każdej iteracji pętli głównej programu losujemy któryś element tej listy (prawdopodobieństwa są tak ustawione, by im wyższe jest średnie prawdopodobieństwo tokenów w danej próbie dowodu tym wyższe jest prawdopodobieństwo wylosowania danej próby) i przy pomocy funkcji `silliest_generate_proof` dopisujemy do tej próby kolejną linię dowodu i jeżeli próby dowodu po dopisaniu tej linii nie ma w liście prób dowodów dopisujemy ją do niej. Przejdźmy teraz do bardziej szczegółowego omówienia implementacji omawianej funjkcji.

`find_proof_little_smarter` jako argumenty przyjmuje obiekt textttbundle typu `NanoGPTBundle`, który odpowiada za generowanie kolejnych tokenów, zmienną tekstową `prompt` która koduje sekwent do udowodnienia w notacji takiej samej jak w `silliest_generate_proof`, zmienną `temperature` typu `float` która służy do regulacji poziomu determinizmu i losowości podczas generowania kolejnych tokenów, oraz zmienną `max_iter` typu `int`, która określa ile maksymalnie iteracji głównej pętli dopuszczamy.

Na początku funkcja parsuje `prompt` do obiektu `goal` klasy `LittleProblem`. Bazowy kontekst `goal` jest przechowywany w liście `base_context` Następnie tworzymy listę `prompts` w którym umieszczamy oryginalny prompt oraz tabelę `probabilities_wild`, która tworzy listę liczb potrzebnych do obliczenia, prawdopodobieństwa losowania każdego elementu `prompts`, które trzymamy w tabeli `probabilities`. W pętli głównej losujemy element `prompts` i przy pomocy generujemy kolejną linię wylosowanej próby do-

wodu. Dopisujemy ją do wylosowanego prompta, a następnie przy pomocy funkcji `extract_proof_part` wyciągamy z tak powstałego prompta do zmiennej `new_proof` część kodującą dowód. Parsujemy dowód do Parsujemy

Obserwacja 1

W logice klasycznej rachunku zdań:

$$((\Gamma_1 \vdash \varphi_1 \quad \wedge \quad \Gamma_2 \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n \vdash \varphi_n) \to \Gamma \vdash \phi)$$

$$\leftrightarrow$$

$$((\Gamma_1, \Gamma \vdash \varphi_1 \quad \wedge \quad \Gamma_2, \Gamma \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n, \Gamma \vdash \varphi_n) \to \Gamma \vdash \phi)$$

Dowód

$(\to)$

Jeżeli $\Gamma_i \vdash \varphi_i$, to $\Gamma_i, \Gamma \vdash \varphi_i$ ponieważ jesteśmy w stanie skonstruować dowód φ_i w kontekście Γ_i, Γ używając jedynie formuł z kontekstu Γ_i. Tak więc

$$(\Gamma_1, \Gamma \vdash \varphi_1 \quad \wedge \quad \Gamma_2, \Gamma \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n, \Gamma \vdash \varphi_n)$$

$$\to$$

$$(\Gamma_1 \vdash \varphi_1 \quad \wedge \quad \Gamma_2 \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n \vdash \varphi_n)$$

$$\to$$

$$\Gamma \vdash \varphi$$

$(\leftarrow)$

Załóżmy nie wprost, że teza nie zachodzi. Wtedy istnieją takie konteksty $\Gamma_1, \Gamma_2, \dots, \Gamma_n, \Gamma$ i formuły $\phi_1, \phi_2, \dots, \phi_n$, że $(\Gamma_1, \Gamma \vdash \varphi_1 \wedge \Gamma_2, \Gamma \vdash \varphi_2 \wedge \dots \wedge \Gamma_n, \Gamma \vdash \varphi_n) \to \Gamma \vdash \phi$ jest prawdziwe oraz, że $(\Gamma_1 \vdash \varphi_1 \wedge \Gamma_2 \vdash \varphi_2 \wedge \dots \wedge \Gamma_n \vdash \varphi_n) \to \Gamma \vdash \phi$ jest fałszywe. Niech

$$F(\Gamma_i) := \text{koninkcja formuł w kontekście } \Gamma_i$$

Z faktu, że w logice klasycznej dla rachunku zdań każde zdanie prawdziwe dla każdego wartościowania jest dowodliwe, to $\Gamma_i \vdash \phi_i$ jest równoważne, z $F(\Gamma_i) \to \varphi_i$.
Fałszywość $(\Gamma_1 \vdash \varphi_1 \wedge \Gamma_2 \vdash \varphi_2 \wedge \dots \wedge \Gamma_n \vdash \varphi_n) \to \Gamma \vdash \phi$ oznacza, że istnieje wartościowanie takie, że dla każdego $i \in \{1, 2, \dots, n\}$ zachodzi $F(\Gamma_i) \to \phi_i$, ale nie zachodzi $F(\Gamma) \to \phi$, ale to oznacza, że $F(\Gamma)$ dla tego wartościowania zachodzi, z kolei ϕ nie zachodzi. Ale ponieważ dla każdego $i \in \{1, 2, \dots, n\}$ zachodzi $F(\Gamma_i) \to \phi_i$ i $F(\Gamma)$ zachodzi to znaczy, że $i \in \{1, 2, \dots, n\}$ zachodzi $(F(\Gamma_i) \wedge F(\Gamma)) \to \phi_i$ i $F(\Gamma)$, czyli zachodzi

$$(\Gamma_1, \Gamma \vdash \varphi_1 \quad \wedge \quad \Gamma_2, \Gamma \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n, \Gamma \vdash \varphi_n) \wedge F(\Gamma)$$

, czyli ϕ zachodzi dla tego wartościowania co daje sprzeczność.